GO 1.25 | POSTGRESQL | DDD / CLEAN ARCHITECTURE | EVENT-DRIVEN

Themis

Developer Onboarding & Architecture Guide — the Phase-3 greenfield platform

Generated 2026-08-17 · repository snapshot at v0.4.1 era, branch `feat/gui-multi-scanner-phase-a` · sources: CLAUDE.md, docs/adr, docs/engineering (EDRs, STACK, CONVENTIONS, PHASE3-STATUS), code inspection

Contents

1. Product overview & core value proposition

Themis is an open-source security-intelligence platform. It ingests the software inventories that modern toolchains already produce (SBOMs in CycloneDX or SPDX, VEX statements, scanner reports), correlates them against live vulnerability feeds, and turns the raw flood of CVE matches into *governed, auditable enterprise decisions* published as standards-based artifacts (OpenVEX, CycloneDX-VEX, CSAF).

The problem it solves

A single container image SBOM can match hundreds of CVEs. Most are noise for a given deployment: fixed by the distro's backport, unreachable in the build, or already accepted as risk. Enterprises need to (a) find the real ones, (b) decide about them *once*, with a recorded reason, and (c) prove to auditors and customers what was decided and why. Themis' pipeline is built around exactly that loop.

What makes it different

- **Deterministic core, advisory AI.** Rules, feeds, and reconciliation are deterministic and auditable. An LLM Gateway can *propose* a position — it can never decide. Every AI proposal enters the same human/policy governance path as any other proposal ("AI proposes, humans/policy decide").
- **"Gathering Is Not Knowing."** Vendor VEX, distro advisories, and feeds are *gathered as information*, never obeyed as truth. A Red Hat "not affected" raises a proposal; only policy (under strict evidence rules) or a human converts it into an Enterprise Position.
- **VEX overlays, never deletes.** Suppression is an overlay with a recorded premise; raw findings are preserved forever. A suppressed finding's *residual priority* drops to 0 while its intrinsic severity is untouched — and a disposition watcher re-opens the question if the premise drifts (CVE enters KEV, EPSS rises, review date passes). An acceptance does not vanish — it expires.
- **Operationally boring, on purpose.** Single static Go binaries, one PostgreSQL server, no external broker, no Node toolchain. Events ride a transactional outbox into a Postgres-backed bus; a real broker

can slot in later behind the same envelope.

> **The one thing to know before touching code:** the repository holds *two code trees*. The legacy PoC monolith (`internal/{domain,usecase,adapter,infrastructure}`, `cmd/themis`) is **frozen at v0.3.x — reference only, never modify**. All new work lands in the Phase-3 greenfield tree (`internal/<context>/{domain,app,adapters}`, `cmd/<context>`). Where an ADR (`docs/adr/`) or EDR (`docs/engineering/decisions/`) constrains a choice, that document wins — over the PoC and over intuition.

2. System architecture & data flow

2.1 The shape: bounded contexts on a pipeline

The platform is a set of independently deployable **bounded-context services**, each a Clean-Architecture stack (`domain ← app ← adapters`, inward-only imports, composition root in `cmd/<context>`). Contexts collaborate **only** through (a) domain events over a transactional outbox + Postgres event bus, and (b) read-only HTTP APIs behind small client seams. No cross-context imports — this is enforced by lint (depguard), go-cleanarch, and a dedicated architecture test suite.

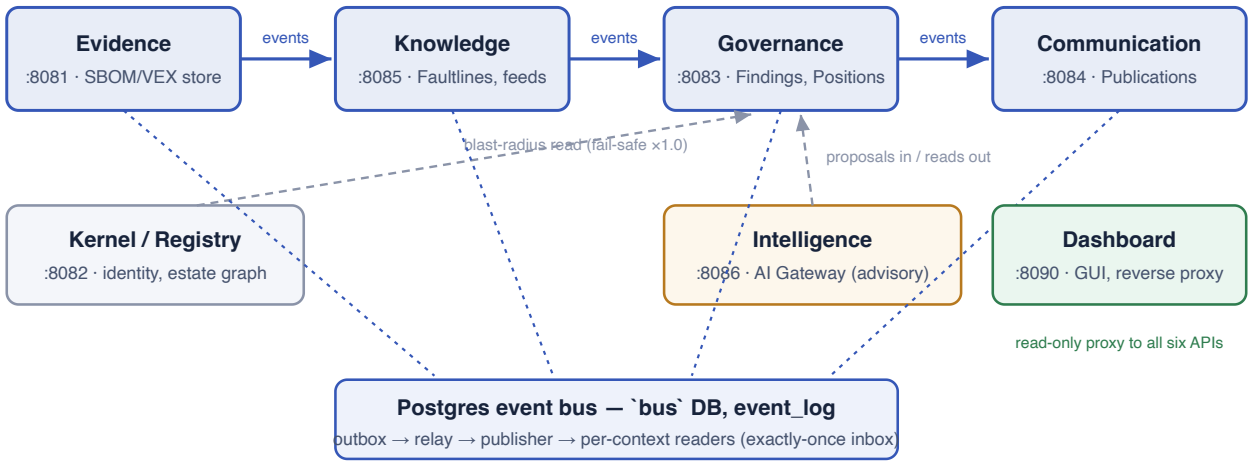

Figure 1 — the pipeline contexts (solid arrows: event flow), the side services, and the Postgres-backed bus. Dashed lines are HTTP read seams or outbox drains; the Dashboard is a view that owns no truth.

2.2 Data flow: the life of one SBOM

1. **Register identity.** Registry records Product → Project → Release (plus the enterprise estate graph Product → Microservice → Deployment → Customer, which later feeds blast-radius).
2. **Upload evidence.** Evidence stores the SBOM (or VEX / curated scanner report) *content-addressed and immutable* — byte-identical re-uploads dedup. It folds documents into a canonical component inventory and emits events.
3. **Correlate.** Knowledge consumes the inventory (via the bus + Evidence's read API), matches components against vulnerability data, and maintains one **Faultline** card per canonical CVE — created once, enriched

forever, never deleted. Opt-in feeds (NVD, EPSS/KEV, Red Hat, Alpine, CSAF-VEX; OSV always on) enrich *existing* cards only — the platform never mirrors a feed.

4. **Govern.** Governance consumes correlation events, creates one **Finding** per (release, CVE), computes triage priority (base score × blast-radius multiplier), and manages append-only Proposals → immutable Enterprise Position versions. Policy may auto-accept only the single constitutional rule (observed `not_affected`); everything else is a human's click.
5. **Publish.** Communication materializes decided positions into deterministic Publications and serializes them (OpenVEX, CycloneDX-VEX, CSAF, markdown, JSON, text) with exactly-once delivery mechanics.
6. **Advise (optional).** Intelligence, when asked, grounds an LLM on the Finding + reconciled ranges + precedent Positions and files a proposal back into Governance — advisory evidence class, never auto-accepted.

> **Mental model:** facts flow forward (Evidence → Knowledge → Governance → Communication) as events; questions flow backward as read-only HTTP reads. Nothing writes into an upstream context, and nothing reaches across a context boundary in-process.

3. Directory structure breakdown

```
themis/
├── cmd/                      # composition roots — one static binary per service
│   ├── registry/  evidence/  knowledge/  governance/  communication/  intelligence/
│   ├── dashboard/            # the GUI: vanilla-JS SPA (go:embed) + reverse proxy
│   ├── authadmin/            # CLI: mint/revoke API keys (bcrypt-hashed at rest)
│   └── themis/               # ⚠ FROZEN v0.3.x monolith — reference only
├── internal/
│   ├── kernel/               # shared value objects, ids, event envelopes (greenfield)
│   ├── registry/             # Product→Project→Release + estate graph
│   ├── evidence/ knowledge/ governance/ communication/ intelligence/
│   │   └── {domain,app,adapters}/     # Clean Architecture rings, inward-only
│   │        └── adapters/store/migrations/   # per-context, reversible SQL
│   ├── dashboard/{proxy,session}/     # GUI server seams (a view, not a context)
│   ├── platform/{observability,eventbus,auth}/  # the ONLY shared packages
│   ├── testutil/
│   └── domain/ usecase/ adapter/ infrastructure/  # ⚠ FROZEN legacy tree
├── api/                      # OpenAPI specs — spec-first; handlers are GENERATED
├── docs/
│   ├── adr/                  # 69 ADRs — binding architectural decisions
│   ├── engineering/          # STACK.md · CONVENTIONS.md · PHASE3-STATUS.md · PARITY-GAP.md
│   │   └── decisions/        # EDR-* — per-context/cross-cutting decision records
│   ├── architecture/         # the architecture book (Books I–III)
│   └── BACKLOG.md            # Part 1 = active tracker · Part 2 = frozen history
├── openspec/changes/         # system of record for greenfield work items
├── scripts/                  # operator/dev drivers (see cheat sheet)
├── deploy/                   # node.env.example (self-documented) + systemd units
├── tests/                    # architecture tests + cross-context pipeline e2e
└── Makefile                  # the quality gates live here
```

Ring	May contain	Must never contain
`domain/`	Aggregates, value objects, invariants — pure Go	I/O, logging, framework imports, SQL
`app/`	Use cases + ports (interfaces); orchestration	Drivers, HTTP, platform packages
`adapters/`	Postgres stores, HTTP handlers, feed ACLs, consumers	Cross-context imports, business rules
`cmd/<ctx>`	Wiring, config from env, lifecycle	Logic of any kind

4. Key components & services

Service	Port	Owns	Know this
Registry (+ Kernel)	:8082	Product→Project→Release identity; estate graph; blast-radius traversal	Co-locates its schema in the `evidence` DB with its own migration bookkeeping. Blast-radius = unique customers a release reaches.
Evidence	:8081	Immutable, content-addressed SBOM/VEX/scanner-report store; canonical inventory	Byte-identical uploads dedup. Discovery runs at upload; a re-discovery sweep re-runs it for stale releases.

Knowledge	:8085	Faultline aggregate (one card per CVE); feed ACLs; correlation	Proposals are append-only; the reconciled view is deterministic by source precedence; lifecycle is forward-only — cards are superseded, never deleted. A distro advisory's package list is *scope, not N claims* (`claim_class` : carrier / scope / unknown-as-carrier).
Governance	:8083	Findings; append-only Proposals; immutable Position versions; triage priority; disposition watcher	Priority = base × blast multiplier (fail-safe ×1.0, saturating ×2.0). Suppression zeroes *residual* priority only, and records its premise for the drift watcher.
Communication	:8084	Publication materialization; 6 serializers; delivery worker	Immutable content + mutable delivery status; append-and-supersede; nothing auto-publishes.
Intelligence	:8086	AI Gateway: `recommend_position` , `plan_remediation` , `explain_vulnerability`	All LLM code confined here behind a provider port; no truth store. Decision outputs go to Governance as proposals; Information outputs are ephemeral. Honest `insufficient` is a feature; a 204's cause rides `X-Themis-AI-Reason` .
Dashboard	:8090	The GUI — vanilla-JS SPA + same-origin reverse proxy + sessions	A view, not a context: no DB, every fact fetched live. The proxy exists because nodes set no CORS and the node API key must never reach browser JS.

Shared platform packages (importable by adapters + cmd only)

- **`platform/observability`** — one `log.Info(...)` tees a console line and an OTel record (Convention R1). Domain/app rings never log.
- **`platform/eventbus`** — business-agnostic outbox publisher, gap-free stream reader, exactly-once consumer inbox over the `bus` database. Moves kernel `Envelope` s only; imports no bounded context.
- **`platform/auth`** — inbound-edge `X-API-Key` auth (bcrypt store, method-based scopes: admin = read+write, read = read-only). Opt-in per node via DSN; `THEMIS_AUTH_REQUIRED=1` makes a missing DSN fail startup.

5. Setup & debugging cheat sheet

First hour

```
# build everything (greenfield + frozen monolith)
go build ./...
# THE quality gate — run before any commit; CI runs `make check-ci`
make check
# read these, in order, before writing code
#   CLAUDE.md → docs/engineering/PHASE3-STATUS.md → docs/BACKLOG.md (Part 1)
#   docs/engineering/{STACK.md,CONVENTIONS.md} → the EDR for your context
```

Running the stack locally

```
# one Postgres server; databases: evidence knowledge governance communication bus auth
intelligence
go build -o bin/ ./cmd/...
# env template (self-documented): deploy/node.env.example
# each node: THEMIS_<CTX>_DATABASE_DSN + THEMIS_<CTX>_MIGRATE=1
# the switch that makes events CROSS contexts (unset = log-only, single-context dev):
export THEMIS_BUS_DATABASE_DSN=postgres://.../bus
# drive an SBOM end to end:
scripts/gf-upload-sbom.sh            # register Product→Project→Release + upload
```

Command	What it does
`make check` / `make check-ci`	Full gate: build · vet-tags · test · lint · clean-arch · arch-test · coverage · deadcode. CI variant scopes coverage to greenfield.
`make test-integration`	`-tags=integration -p 1`, real embedded Postgres
`make vet-tags`	Type-checks every build tag (integration/e2e/llm/postgres) — tagged files are invisible to plain builds and rot silently
`make e2e-pipeline`	SBOM → published OpenVEX across all contexts over the real bus
`make generate-api-<ctx>`	Regenerate handlers after editing `api/<ctx>.openapi.yaml` — never hand-edit generated code
`go test -run TestX ./internal/<ctx>/...`	Single test (add `-tags=integration` when needed)
`scripts/vm-verify.sh`	One read-only deployment report: services, migrations, pipeline counts, reader lag, feeds
`scripts/release-posture.sh`	Consolidated release posture from the CLI (add `--ai N` for recommendations)
`go run ./cmd/authadmin create-key --name dev --scopes admin`	Mint an API key (token printed once; only its bcrypt hash is stored)

Debugging gotchas (each one cost somebody a day)

- **Events not crossing contexts?** `THEMIS_BUS_DATABASE_DSN` is unset — the publisher is log-only and readers are disabled. Set it on *every* node.

- **A node "works" but its tagged tests don't compile?** Run `make vet-tags`. A `//go:build` file is invisible to normal builds; the repo's only real-model test was once dead for days this way.
- **AI calls timing out at ~60s despite a raised LLM timeout?** Three deadlines govern one recommendation and *the shortest wins*: raise `THEMIS_INTELLIGENCE_TIMEOUT` (Governance side) *and* `THEMIS_LLM_TIMEOUT` (Intelligence side) together — a caller hang-up is logged as `provider_error` and misread as an AI fault.
- **AI returned 204?** Read `X-Themis-AI-Reason`: `disabled` · `unreachable` · `insufficient` (the model honestly declined — the seam working) · `provider_error` · `business_invalid`.
- **Re-upload "does nothing"?** Evidence is content-addressed; byte-identical uploads dedup by design. A re-run needs changed bytes.
- **Node refuses to boot with auth errors?** `THEMIS_AUTH_REQUIRED=1` hard-fails startup when the auth DSN is empty — that's the production guard working, not a bug.
- **clean-arch fails on a test file?** go-cleanarch scans tests too — a cross-layer test in the wrong ring directory fails the gate.
- **Coverage gate fails on a new package?** Packages must be registered in `scripts/check-coverage.sh` (domain/app 100% · adapters 90% · stores 80%).
- **Feed enrichment silent?** All feeds except OSV are opt-in (`THEMIS_NVD_ENABLED=1`, `THEMIS_EPSSKEV_ENABLED=1`, `THEMIS_REDHAT_ENABLED=1`, …) — no silent outbound calls, ever.
- **Dashboard curl returns 303/401?** With auth on, *all* static assets sit behind the session gate. Verify a deploy offline with `grep -ac <marker> bin/dashboard` instead.

House rules that override habit

- Never modify the frozen legacy tree. Never commit/push without an explicit ask. PR creation/merge is deny-listed tooling, deliberately.
- ADR/EDR wins over intuition; OpenSpec (`openspec/changes/phase3-*`) is the system of record for greenfield work.
- New packages, dependencies, API or domain-model changes require a stated why/alternatives/impact *before* code.
